

# ДИЗАЙН ТА РЕАЛІЗАЦІЯ RESTFUL API

Питання 4.2.

The Art and Science of Full-Stack Development: From  
Concept to Cloud

(Volume 3)



AUTHOR: NNAEMEKA KINGSLEY UGWUMBA  
(First Edition)

All Rights Reserved 2025

# RESTful API

---

- **REST (Representational State Transfer)** – архітектурний стиль для проєктування додатків з мережевою підтримкою, що став типовим для Web API.
  - RESTful API використовує протоколи HTTP та слідує конкретним обмеженням, щоб створити масштабовані, безстанові та кешовані вебсервіси.
- Основні архітектурні обмеження для REST було сформульовано в дисертації Роя Філдінга (2000):
  - Клієнт-серверна архітектура, відокремлення відповідальностей між UI та сховищем даних;
  - Безстановість: кожен запит містить усю необхідну інформацію для обробки, стан сесії не зберігається на сервері;
  - Кешованість: відгуки повинні самовизначатись як кешовані або некешовані, щоб покращити ефективність обміну даних у мережі;
  - Шарувата система: архітектура komponується з ієрархічних рівнів, де кожен компонент не є видимим з інших рівнів, крім наступного;
  - Код за вимогою (необов'язково): сервери можуть розширити клієнтську функціональність, передаючи виконуваний код;
  - Узгоджений інтерфейс (метод) взаємодії між компонентами, зокрема, ідентифікації ресурсів у запитах, оперування ресурсами через представлення (representations), самоописові повідомлення та HATEOAS.

# RESTful API

```
// Приклади коректного іменування RESTful-ресурсів
const resourceEndpoints = {
  // Колекції ресурсів (іменники у множині)
  users: '/api/users', // Колекція користувачів
  tasks: '/api/tasks', // Колекція завдань
  projects: '/api/projects', // Колекція проєктів

  // Одиничні ресурси (однина для унікальних сутностей)
  profile: '/api/profile', // Профіль поточного користувача
  settings: '/api/settings', // Налаштування користувача

  // Підресурси (ієрархічні зв'язки)
  userTasks: '/api/users/:userId/tasks', // Завдання, що належать користувачу
  projectMembers: '/api/projects/:projectId/members', // Учасники проєкту

  // Дії (дієслова для не-CRUD операцій)
  login: '/api/auth/login', // Дія автентифікації
  logout: '/api/auth/logout', // Завершення сесії
  search: '/api/search', // Пошук по ресурсах
  import: '/api/data/import', // Масовий імпорт
  export: '/api/data/export', // Експорт даних
};
```

```
// Антипатерни, яких слід уникати
const antiPatterns = {
  verbsInResourceNames: '/api/getUsers', // Замість цього слід використовувати GET /api/users
  actionsOnCollections: '/api/users/create', // Замість цього слід використовувати POST /api/users
  fileExtensions: '/api/users.json', // Слід використовувати заголовок Content-Type
  queryParamsForActions: '/api/users?action=delete', // Слід використовувати DELETE /api/users/:id
};
```

- REST API – це ресурсно-орієнтована система, кожен ресурс має URI, а взаємодія відбувається через HTTP-методи.

```
// Відображення HTTP-методів у REST
const httpMethodSemantics = {
  GET: {
    purpose: 'Retrieve resource(s)',
    idempotent: true,
    safe: true,
    responseCodes: [200, 404],
    examples: [
      'GET /api/users', // Отримати список користувачів
      'GET /api/users/123', // Отримати конкретного користувача
      'GET /api/users?role=admin', // Відфільтрувати користувачів
    ],
  },
  POST: {
    purpose: 'Create new resource',
    idempotent: false,
    safe: false,
    responseCodes: [201, 400, 409],
    examples: [
      'POST /api/users', // Створити користувача
      'POST /api/users/123/tasks', // Створити завдання для користувача
    ],
  },
};
```

# RESTful API

---

```
PUT: {
  purpose: 'Replace entire resource',
  idempotent: true,
  safe: false,
  responseCodes: [200, 201, 204, 400],
  examples: [
    'PUT /api/users/123', // Повністю замінити дані користувача
    'PUT /api/settings', // Оновити налаштування
  ],
},

PATCH: {
  purpose: 'Partial resource update',
  idempotent: false,
  safe: false,
  responseCodes: [200, 204, 400],
  examples: [
    'PATCH /api/users/123', // Частково оновити поля користувача
    'PATCH /api/tasks/456', // Позначити завдання як виконане
  ],
},

DELETE: {
  purpose: 'Remove resource',
  idempotent: true,
  safe: false,
  responseCodes: [200, 204, 404],
  examples: [
    'DELETE /api/users/123', // Видалити користувача
    'DELETE /api/tasks/456', // Видалити завдання
  ],
},
```

```
HEAD: {
  purpose: 'Retrieve headers only',
  idempotent: true,
  safe: true,
  responseCodes: [200, 404],
  examples: [
    'HEAD /api/users/123', // Перевірити, чи існує користувач
  ],
},

OPTIONS: {
  purpose: 'Discover allowed methods',
  idempotent: true,
  safe: true,
  responseCodes: [200],
  examples: [
    'OPTIONS /api/users', // Отримати підтримувані методи
  ],
},
};
```

## Зразок повної REST-реалізації (метод listTasks())

```
const express = require('express');
const router = express.Router({ mergeParams: true });
// Контролер ресурсу Task
class TaskController {
  constructor(taskService) {
    this.taskService = taskService;
  }
  // GET /api/tasks - отримати список завдань із фільтрацією,
  сортуванням і пагінацією
  async listTasks(req, res, next) {
    try {
      const {
        page = 1, limit = 20, sortBy = 'createdAt',
        sortOrder = 'desc', status, priority, assigneeId,
        projectId, search,
      } = req.query;

      const filter = {
        ...(status && { status }),
        ...(priority && { priority }),
        ...(assigneeId && { assigneeId }),
        ...(projectId && { projectId }),
        ...(search && {
          $or: [
            { title: { $regex: search, $options: 'i' } },
            { description: { $regex: search, $options: 'i' } },
          ],
        }),
      };

      const result = await this.taskService.findTasks({
        filter,
        pagination: {
          page: parseInt(page),
          limit: parseInt(limit),
        },
        sort: { [sortBy]: sortOrder === 'desc' ? -1 : 1, },
      });
```

```
// Додаємо HATEOAS-посилання
const baseUrl = `${req.protocol}://${req.get('host')}${req.baseUrl}`;
const links = {
  self: { href: `${baseUrl}?${new URLSearchParams(req.query)}` },
  first: { href: `${baseUrl}?page=1&limit=${limit}` },
  prev:
    page > 1
      ? { href: `${baseUrl}?page=${page - 1}&limit=${limit}` } : null,
  next:
    page < result.totalPages
      ? { href: `${baseUrl}?page=${page + 1}&limit=${limit}` } : null,
  last: { href: `${baseUrl}?page=${result.totalPages}&limit=${limit}` },
};

res.json({
  success: true, data: result.tasks,
  meta: {
    pagination: {
      page: result.page, limit: result.limit,
      totalItems: result.totalItems, totalPages: result.totalPages,
      hasNext: result.page < result.totalPages,
      hasPrev: result.page > 1,
    },
    filter,
    sort: { sortBy, sortOrder },
  },
  links,
});
} catch (error) { next(error); }
```

```
// POST /api/tasks - створити нове завдання
```

```
async createTask(req, res, next) {  
  try {  
    const taskData = {  
      ...req.body,  
      createdBy: req.user.id,  
      createdAt: new Date(),  
    };  
  
    const task = await this.taskService.createTask(taskData);  
    const taskUrl =  
`${req.protocol}://${req.get('host')}${req.baseUrl}/${task.id}`;  
  
    res  
      .status(201)  
      .location(taskUrl)  
      .json({  
        success: true,  
        data: task,  
        links: {  
          self: { href: taskUrl },  
          parent: {  
            href:  
`${req.protocol}://${req.get('host')}${req.baseUrl}`,  
          },  
        },  
      });  
  } catch (error) {  
    if (error.name === 'ValidationError') {  
      error.statusCode = 400;  
    }  
    next(error);  
  }  
}
```

## Зразок повної REST-реалізації (метод createTask())

- Метод listTasks() демонструє одразу кілька REST-практик: зчитування query-параметрів, формування об'єкта filter, параметрів пагінації та сортування, а також повернення HATEOAS-посилань self, first, prev, next, last.
  - Відповідь містить не лише дані, а й навігаційний контекст для клієнта.
- Метод createTask() реалізує створення нового ресурсу коректно з точки зору HTTP: додає службові поля createdBy і createdAt, викликає сервіс створення, встановлює статус 201 Created, заголовок Location і повертає посилання на щойно створений ресурс.
  - Клієнт одразу отримує канонічну адресу нового об'єкта.

# Зразок повної REST-реалізації (метод `getTask()`)

Методи `getTask()`, `replaceTask()`, `updateTask()` і `deleteTask()` показують типовий REST-цикл роботи з одиничним ресурсом.

- Якщо запис не знайдено, повертається 404 з кодом `TASK_NOT_FOUND`.
- Якщо знайдено, сервер або повертає його представлення, або оновлює, або видаляє.

```
// GET /api/tasks/:taskId - отримати конкретне завдання
async getTask(req, res, next) {
  try {
    const task = await
this.taskService.getTaskById(req.params.taskId);

    if (!task) {
      return res.status(404).json({
        success: false,
        error: {
          message: 'Task not found',
          code: 'TASK_NOT_FOUND',
        },
      });
    }

    const baseUrl =
`${req.protocol}://${req.get('host')}${req.baseUrl}`;

    res.json({
      success: true,
      data: task,
      links: {
        self: { href: `${baseUrl}/${task.id}` },
        parent: { href: baseUrl },
        subtasks: { href: `${baseUrl}/${task.id}/subtasks` },
        comments: { href: `${baseUrl}/${task.id}/comments` },
        attachments: { href:
`${baseUrl}/${task.id}/attachments` },
      },
    });
  } catch (error) {
    next(error);
  }
}
```



# Зразок повної REST-реалізації (методи replaceTask() та updateTask())

```
// PUT /api/tasks/:taskId - повністю замінити завдання
async replaceTask(req, res, next) {
  try {
    const taskData = {
      ...req.body,
      updatedBy: req.user.id,
      updatedAt: new Date(),
    };

    const task = await
this.taskService.replaceTask(req.params.taskId, taskData);

    if (!task) {
      return res.status(404).json({
        success: false,
        error: {
          message: 'Task not found',
          code: 'TASK_NOT_FOUND',
        },
      });
    }

    res.json({
      success: true,
      data: task,
      message: 'Task replaced successfully',
    });
  } catch (error) {
    next(error);
  }
}
```

```
// PATCH /api/tasks/:taskId - частково оновити завдання
async updateTask(req, res, next) {
  try {
    const updates = {
      ...req.body,
      updatedBy: req.user.id,
      updatedAt: new Date(),
    };

    const task = await this.taskService.updateTask(req.params.taskId, updates);

    if (!task) {
      return res.status(404).json({
        success: false,
        error: {
          message: 'Task not found',
          code: 'TASK_NOT_FOUND',
        },
      });
    }

    res.json({
      success: true,
      data: task,
      message: 'Task updated successfully',
    });
  } catch (error) {
    next(error);
  }
}
```

- Показовою є відмінність між PUT і PATCH: у першому випадку формується повний новий стан ресурсу, у другому – лише набір змін.



# Зразок повної REST-реалізації (методи deleteTask() та listSubtasks())

---

```
// DELETE /api/tasks/:taskId - видалити завдання
async deleteTask(req, res, next) {
  try {
    const deleted = await
this.taskService.deleteTask(req.params.taskId);

    if (!deleted) {
      return res.status(404).json({
        success: false,
        error: {
          message: 'Task not found',
          code: 'TASK_NOT_FOUND',
        },
      });
    }

    res.status(204).send();
  } catch (error) {
    next(error);
  }
}
```

```
// GET /api/tasks/:taskId/subtasks - приклад вкладеного ресурсу
async listSubtasks(req, res, next) {
  try {
    const subtasks = await this.taskService.getSubtasks(req.params.taskId);
    const baseUrl =
`${req.protocol}://${req.get('host')}${req.baseUrl}/${req.params.taskId}`;

    res.json({
      success: true,
      data: subtasks,
      links: {
        self: { href: `${baseUrl}/subtasks` },
        parent: {
          href:
`${req.protocol}://${req.get('host')}${req.baseUrl}/${req.params.taskId}`,
        },
      },
    });
  } catch (error) {
    next(error);
  }
}
```

# Зразок повної REST-реалізації

- Приклад переносить валідацію безпосередньо в маршрутний прошарок.
  - validateTask перевіряє заголовок, опис, пріоритет, дату і assigneeId.
  - validateTaskId окремо перевіряє формат ідентифікатора завдання.

```
// Маршрути Task із middleware валідації
const { body, param, query, validationResult } = require('express-validator');
const taskController = new TaskController(taskService);

// Middleware валідації
const validateTask = [
  body('title')
    .trim().notEmpty().withMessage('Title is required')
    .isLength({ max: 255 }).withMessage('Title must be less than 255 characters'),

  body('description')
    .optional().isString().withMessage('Description must be a string')
    .isLength({ max: 2000 }).withMessage('Description must be less than 2000 characters'),

  body('priority')
    .optional()
    .isIn(['low', 'medium', 'high']).withMessage('Priority must be low, medium, or high'),

  body('dueDate')
    .optional().isISO8601().withMessage('Due date must be a valid ISO 8601 date'),

  body('assigneeId')
    .optional().isMongoId().withMessage('Assignee ID must be a valid MongoDB ID'),

  (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({
        success: false,
        errors: errors.array(),
      });
    }
    next();
  },
];
```

```
const validateTaskId = [
  param('taskId')
    .isMongoId().withMessage('Task ID must be a valid MongoDB ID'),

  (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({
        success: false,
        errors: errors.array(),
      });
    }
    next();
  },
];
```

# Зразок повної REST-реалізації

```
// Опис маршрутів
router
  .route('/')
  .get(
    [
      query('page').optional().isInt({ min: 1 }).toInt(),
      query('limit').optional().isInt({ min: 1, max: 100 }).toInt(),
      query('sortBy').optional().isIn(['title', 'priority', 'dueDate',
'createdAt', 'updatedAt']),
      query('sortOrder').optional().isIn(['asc', 'desc']),
      query('status').optional().isIn(['pending', 'in-progress',
'completed', 'archived']),
      query('priority').optional().isIn(['low', 'medium', 'high']),
      query('search').optional().isString().trim().escape(),
    ],
    taskController.listTasks.bind(taskController)
  )
  .post(
    validateTask,
    taskController.createTask.bind(taskController)
  );
```

- Для PATCHN додатково контролюється перелік дозволених полів оновлення.
  - Так REST-інтерфейс відхиляє некоректний запит ще до входу в бізнес-логіку.

08.04.2026

@Марченко С.В., ЧДБК

```
router
  .route('/:taskId')
  .get(
    validateTaskId,
    taskController.getTask.bind(taskController)
  )
  .put(
    validateTaskId,
    validateTask,
    taskController.replaceTask.bind(taskController)
  )
  .patch(
    validateTaskId,
    [
      body().custom((body) => {
        const allowedUpdates = [
          'title', 'description', 'priority', 'status',
          'dueDate', 'assigneeId',
        ];
        const updates = Object.keys(body);
        const isValidOperation = updates.every((update) =>
          allowedUpdates.includes(update)
        );

        if (!isValidOperation) {
          throw new Error('Invalid update fields');
        }

        return true;
      }),
    ],
    taskController.updateTask.bind(taskController)
  )
  .delete(
    validateTaskId,
    taskController.deleteTask.bind(taskController)
  );
```

# Зразок повної REST-реалізації

---

```
// Маршрут вкладеного ресурсу
router.get(
  '/:taskId/subtasks',
  validateTaskId,
  taskController.listSubtasks.bind(taskController)
);

module.exports = router;
```

- Вкладений маршрут `/:taskId/subtasks` демонструє підресурсну модель.
  - Це не окремий плоский endpoint, а ресурс, контекстно пов'язаний із батьківським task.
  - Така побудова маршруту добре узгоджується з ресурсною ієрархією.
- У підсумку продемонстровано повноцінну REST-структуру: контролер, маршрути, валідацію, HTTP-статуси, HATEOAS і вкладені ресурси.

# Стратегії версіонування API

---

```
// Versioning through URL path
const apiV1Router = express.Router();
const apiV2Router = express.Router();

// Маршрути API v1
apiV1Router.get('/tasks', (req, res) => {
  res.json({ version: 'v1', data: [] });
});

// Маршрути API v2 (зміна формату відповіді)
apiV2Router.get('/tasks', (req, res) => {
  // Новий формат відповіді
  res.json({
    version: 'v2',
    tasks: [],
    metadata: { count: 0 },
  });
});

// Підключення версій до застосунку
app.use('/api/v1', apiV1Router);
app.use('/api/v2', apiV2Router);
```

- Версіонування API є ключовим механізмом забезпечення еволюції системи без порушення зворотної сумісності.
  - У контексті REST це означає, що зміни у форматі відповіді, структурі ресурсу або поведінці сервера не повинні ламати вже існуючих клієнтів.
  - Вибір стратегії версіонування безпосередньо впливає на архітектуру системи, механізми кешування, маршрутизацію запитів і складність підтримки.
- Найбільш поширеною є стратегія версіонування через URI, коли номер версії включається безпосередньо в шлях ресурсу, наприклад `/api/v1/users` або `/api/v2/users`.
  - Такий підхід є максимально прозорим для розробників і добре підтримується інфраструктурою, зокрема кешами та CDN, оскільки різні версії мають різні URL і розглядаються як окремі ресурси.
  - Це спрощує маршрутизацію на рівні вебфреймворку та API Gateway.
  - Водночас цей підхід призводить до дублювання інтерфейсів і поступового накопичення технічного боргу, оскільки кожна нова версія фактично створює новий API, який необхідно підтримувати паралельно зі старими.

# Стратегії версіонування API

---

```
// Versioning через HTTP-заголовки (чистіші URL)
app.use('/api/tasks', (req, res, next) => {
  const apiVersion = req.get('Accept-Version') || 'v1';

  if (apiVersion === 'v1') {
    // Логіка v1
    return res.json({ version: 'v1', data: [] });
  } else if (apiVersion === 'v2') {
    // Логіка v2
    return res.json({ version: 'v2', tasks: [] });
  }

  next(new Error('Unsupported API version'));
});
```

- Альтернативою є версіонування через HTTP-заголовки, коли номер версії передається, наприклад, у заголовок Accept-Version.
  - У цьому випадку URI залишається стабільним, а версія інтерпретується як характеристика представлення ресурсу, а не самого ресурсу.
  - Це краще відповідає REST-принципам, оскільки ресурс і його представлення розділяються концептуально.
  - Однак така стратегія ускладнює використання кешів, оскільки стандартні механізми кешування орієнтуються на URL і не враховують заголовки без додаткової конфігурації (наприклад, через Vary).
  - Крім того, цей підхід підвищує складність клієнтських реалізацій і тестування.

# Стратегії версіонування API

---

```
// Versioning через content negotiation
app.get('/api/tasks', (req, res) => {
  const acceptHeader = req.get('Accept') || '';

  if (acceptHeader.includes('application/vnd.taskflow.v2+json')) {
    return res.json({ version: 'v2', tasks: [] });
  }

  // За замовчуванням v1
  res.json({ version: 'v1', data: [] });
});
```

Більш формалізованим варіантом є версіонування через content negotiation, коли версія кодується у MIME-типі, наприклад `application/vnd.company.v2+json`.

- Такий підхід є найбільш канонічним з точки зору REST, оскільки повністю спирається на стандартні механізми HTTP і дозволяє гнучко керувати різними представленнями одного ресурсу.
- Він доречний у системах, де один і той самий ресурс може мати різні формати (наприклад, JSON, XML, або різні схеми даних).
- Водночас цей підхід є складним у реалізації та підтримці, а також менш зручним для більшості клієнтів, тому на практиці використовується значно рідше.



# Стратегії версіонування API

---

```
// Middleware для керування версіями
function versionTransition(req, res, next) {
  const requestedVersion = req.get('X-API-Version') || 'v1';
  const supportedVersions = ['v1', 'v2'];

  if (!supportedVersions.includes(requestedVersion)) {
    return res.status(400).json({
      error: `Unsupported API version. Supported versions:
${supportedVersions.join(', ')}`
    });
  }

  req.apiVersion = requestedVersion; // Зберігаємо версію в запиті
  res.set('X-API-Version', requestedVersion); // Додаємо версію в відповідь

  next();
}

// Приклад із попередженням про застарілу версію
app.get('/api/v1/tasks', versionTransition, (req, res) => {
  if (req.apiVersion === 'v1') {
    // Попередження про застарілість API
    res.set('Warning', '299 - "v1 is deprecated. Migrate to v2 by 2024-12-31."');
    res.set('Sunset', 'Wed, 31 Dec 2024 23:59:59 GMT');
  }

  res.json({ data: [] });
});
```

- Окрему категорію становить версіонування на рівні middleware або API Gateway, де версія визначається централізовано (наприклад, через заголовки, параметри або навіть доменні імена) і далі передається у внутрішній pipeline обробки запиту.
  - Це дозволяє ізолювати логіку версіонування від бізнес-логіки та забезпечити більш гнучке керування версіями, включаючи можливість поступового мігрування клієнтів, A/B-тестування або використання feature flags.
  - Такий підхід характерний для складних багаторівневих систем і мікросервісної архітектури, але водночас підвищує складність трасування та відлагодження запитів.

# Контроль інтенсивності запитів

---

```
const rateLimit = require('express-rate-limit');
const RedisStore = require('rate-limit-redis');
const Redis = require('ioredis');

// Підключення до Redis (для розподіленого rate limiting)
const redisClient = new Redis({
  host: process.env.REDIS_HOST,
  port: process.env.REDIS_PORT,
});

// Базовий rate limiter
const basicLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 хвилин
  max: 100, // максимум 100 запитів з IP
  standardHeaders: true, // включає RateLimit-* заголовки
  legacyHeaders: false,
  message: {
    success: false,
    error: {
      message: 'Too many requests, please try again later',
      code: 'RATE_LIMIT_EXCEEDED',
    },
  },
});
```

- Контроль інтенсивності запитів – невід’ємна частина побудови стабільного API.
  - Основна мета полягає не лише в захисті від зловмисних атак, а й у забезпеченні передбачуваного навантаження на систему, особливо в умовах масштабування та багатокористувацького доступу.
- Перший підхід реалізується через `express-rate-limit` і представляє типовий механізм обмеження кількості запитів у заданому часовому вікні.
  - Конфігурація визначає тривалість вікна (`windowMs`) і максимальну кількість запитів (`max`).
  - Включені стандартні HTTP-заголовки (`RateLimit-*`) дозволяють клієнтам отримувати інформацію про поточний стан ліміту.
  - Такий підхід є простим у реалізації, але обмежений у контексті горизонтального масштабування, оскільки стан лічильників за замовчуванням зберігається в пам’яті одного процесу.

# Контроль інтенсивності запитів

```
// Rate limiter з Redis (для масштабованих систем)
const distributedLimiter = rateLimit({
  store: new RedisStore({
    sendCommand: (...args) => redisClient.call(...args),
  }),
  windowMs: 60 * 1000, // 1 хвилина
  max: 50,
  standardHeaders: true,
  legacyHeaders: false,
});

// Динамічний rate limiting залежно від користувача
const dynamicLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: (req) => {
    // Різні ліміти для різних типів користувачів
    if (req.user?.role === 'admin') return 1000;
    if (req.user?.role === 'premium') return 200;
    return 50; // звичайні користувачі
  },
  keyGenerator: (req) => {
    // Ідентифікація користувача
    return req.user?.id || req.ip;
  },
});

// Застосування limiter-ів
app.use('/api', basicLimiter);
app.use('/api/heavy', distributedLimiter);
app.use('/api', dynamicLimiter);
```

- Для вирішення цієї проблеми вводиться розподілений rate limiting із використанням Redis.
  - Тоді всі інстанси застосунку використовують спільне сховище, що забезпечує консистентність обмежень незалежно від кількості серверів.
  - Це критично для систем із балансуванням навантаження, де запити одного користувача можуть оброблятися різними нодами.
  - Redis виступає як централізований механізм координації лімітів.
- Динамічний rate limiting адаптує обмеження залежно від ролі користувача.
  - Тут замість статичного значення max використовується функція, що дозволяє встановлювати різні квоти для адміністратора, преміум-користувача та звичайного користувача.
  - Функція keyGenerator() визначає, як ідентифікується клієнт: за IP або за ідентифікатором користувача.
  - Це дозволяє перейти від грубого обмеження за IP до більш точного контролю на рівні облікового запису.

# Концепція throttling

---

```
// Custom throttling middleware (token bucket)
class TokenBucket {
  constructor(capacity, refillRate) {
    this.capacity = capacity; // максимальна кількість токенив
    this.tokens = capacity;
    this.refillRate = refillRate; // токенив за секунду
    this.lastRefill = Date.now();
  }

  refill() {
    const now = Date.now();
    const timePassed = (now - this.lastRefill) / 1000;
    const refillTokens = timePassed * this.refillRate;
    this.tokens = Math.min(this.capacity, this.tokens + refillTokens);
    this.lastRefill = now;
  }

  consume(tokens = 1) {
    this.refill();

    if (this.tokens >= tokens) {
      this.tokens -= tokens;
      return true;
    }

    return false;
  }
}
```

- Тротлинг відрізняється від rate limiting тим, що не просто обмежує кількість запитів у вікні, а контролює швидкість їх обробки в часі.
  - Реалізація через алгоритм token bucket моделює систему, де кожен користувач має певну “ємність” запитів (burst capacity), яка поступово відновлюється з часом.
  - Метод refill() додає токени відповідно до швидкості поповнення, а consume() перевіряє, чи достатньо ресурсів для обробки запиту.
  - Якщо ні, сервер повертає статус 429.
  - Такий підхід є більш гнучким, оскільки дозволяє короточасні сплески активності, але обмежує довготривале перевантаження.

# Концепція throttling

---

```
// Зберігання bucket-ів (у пам'яті)
const buckets = new Map();
// Middleware throttling
function throttlingMiddleware(capacity, refillRate) {
  return (req, res, next) => {
    const key = req.user?.id || req.ip;

    if (!buckets.has(key)) {
      buckets.set(key, new TokenBucket(capacity, refillRate));
    }

    const bucket = buckets.get(key);

    if (!bucket.consume()) {
      return res.status(429).json({
        success: false,
        error: {
          message: 'Too many requests',
          code: 'THROTTLED',
        },
      });
    }

    next();
  };
}

// Використання throttling
app.use('/api/stream', throttlingMiddleware(10, 5)); // burst 10, refill 5/sec
```

- Використання Map для зберігання bucket-ів демонструє просту реалізацію в пам'яті, однак у реальних системах цей механізм також потребує зовнішнього сховища (наприклад, Redis) для забезпечення консистентності між інстансами.
- У підсумку, ефективне обмеження запитів є багаторівневим:
  - базовий rate limiting забезпечує загальний контроль,
  - розподілений механізм гарантує масштабованість,
  - динамічні правила враховують бізнес-логіку,
  - throttling дозволяє тонко керувати навантаженням у часі.

# Специфікація OpenAPI

---

```
// Специфікація OpenAPI
const swaggerOptions = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'TaskFlow API',
      version: '1.0.0',
      description: 'Task management API documentation',
      contact: {
        name: 'API Support',
        email: 'support@taskflow.app',
      },
      license: {
        name: 'MIT',
        url: 'https://opensource.org/licenses/MIT',
      },
    },
  },

  servers: [
    {
      url: 'http://localhost:3000/api',
      description: 'Development server',
    },
    {
      url: 'https://api.taskflow.app',
      description: 'Production server',
    },
  ],
}
```

- Блок info містить метадані про API: назву, версію, опис, контактні дані та ліцензію.
- Блок servers задає середовища виконання, окремо для локальної розробки і продакшну.
  - Це важливо не лише для довідки, а й для коректної роботи інтерфейсу Swagger UI, який повинен знати базові адреси API.

# Специфікація OpenAPI

- Блок **components** є основою повторного використання в OpenAPI.
  - У **securitySchemes** описуються дві схеми автентифікації: JWT через bearer token і API key через заголовок X-API-Key.
  - Тут правила безпеки не дублюються вручну в кожному endpoint, а задаються централізовано і можуть багаторазово підключатися.
  - У **schemas** описуються моделі Task і Error, тобто структури успішної й помилкової відповіді.
  - Тут зафіксовані як типи полів, так і допустимі значення (enum), приклади (example) та формати (date-time).
  - Така документація не просто “для читання”, а формалізований контракт, який може бути використаний генераторами клієнтів, тестовими інструментами і валідаторами.

```
components: {
  securitySchemes: {
    BearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT', },
    ApiKeyAuth: { type: 'apiKey', in: 'header', name: 'X-API-Key', },
  },

  schemas: {
    Task: {
      type: 'object', required: ['title', 'status'],
      properties: {
        id: { type: 'string', description: 'Task identifier', example: '507f1f77bcf86cd799439011', },
        title: { type: 'string', description: 'Task title', example: 'Complete API documentation', },
        description: { type: 'string', description: 'Detailed task description',
          example: 'Write comprehensive API documentation with examples',
        },
        status: {
          type: 'string', enum: ['pending', 'in-progress', 'completed', 'archived'],
          example: 'in-progress',
        },
        priority: { type: 'string', enum: ['low', 'medium', 'high'], example: 'high', },
        dueDate: { type: 'string', format: 'date-time', example: '2023-12-31T23:59:59Z', },
      },
    },

    Error: {
      type: 'object',
      properties: {
        success: { type: 'boolean', example: false, },
        error: {
          type: 'object',
          properties: {
            message: { type: 'string', example: 'Task not found', },
            code: { type: 'string', example: 'TASK_NOT_FOUND', },
            details: {
              type: 'array',
              items: { type: 'object', },
            },
          },
        },
      },
    },
  },
},
```



# Специфікація OpenAPI

```
parameters: {
  TaskId: {
    name: 'taskId', in: 'path', required: true,
    description: 'ID of the task',
    schema: {
      type: 'string',
      example: '507f1f77bcf86cd799439011',
    },
  },
  PaginationPage: {
    name: 'page', in: 'query', required: false,
    description: 'Page number for pagination',
    schema: {
      type: 'integer',
      minimum: 1,
      default: 1,
    },
  },
  PaginationLimit: {
    name: 'limit', in: 'query', required: false,
    description: 'Number of items per page',
    schema: {
      type: 'integer',
      minimum: 1,
      maximum: 100,
      default: 20,
    },
  },
}
```

```
responses: {
  NotFound: {
    description: 'Resource not found',
    content: {
      'application/json': {
        schema: {
          $ref: '#/components/schemas/Error',
        },
        example: {
          success: false,
          error: { message: 'Resource not found', code: 'NOT_FOUND' },
        },
      },
    },
  },
  Unauthorized: {
    description: 'Authentication required',
    content: {
      'application/json': {
        schema: { $ref: '#/components/schemas/Error' },
        example: {
          success: false,
          error: {
            message: 'Authentication required', code: 'AUTH_REQUIRED' },
        },
      },
    },
  },
},
security: [
  { BearerAuth: [], },
],
apis: ['./routes/*.js', './models/*.js'], // Шляхи до файлів API
};
```

- повторно використовувані параметри, описані в компонентах TaskId, PaginationPage та PaginationLimit, дозволяють не переписувати однакові query- та path-параметри для кожного маршруту.
- Аналогічно, готові відповіді NotFound і Unauthorized стандартизують структуру типових помилок.



# Базова документація в Swagger

```
const express = require('express');
const swaggerJsdoc = require('swagger-jsdoc');
const swaggerUi = require('swagger-ui-express');

const app = express();

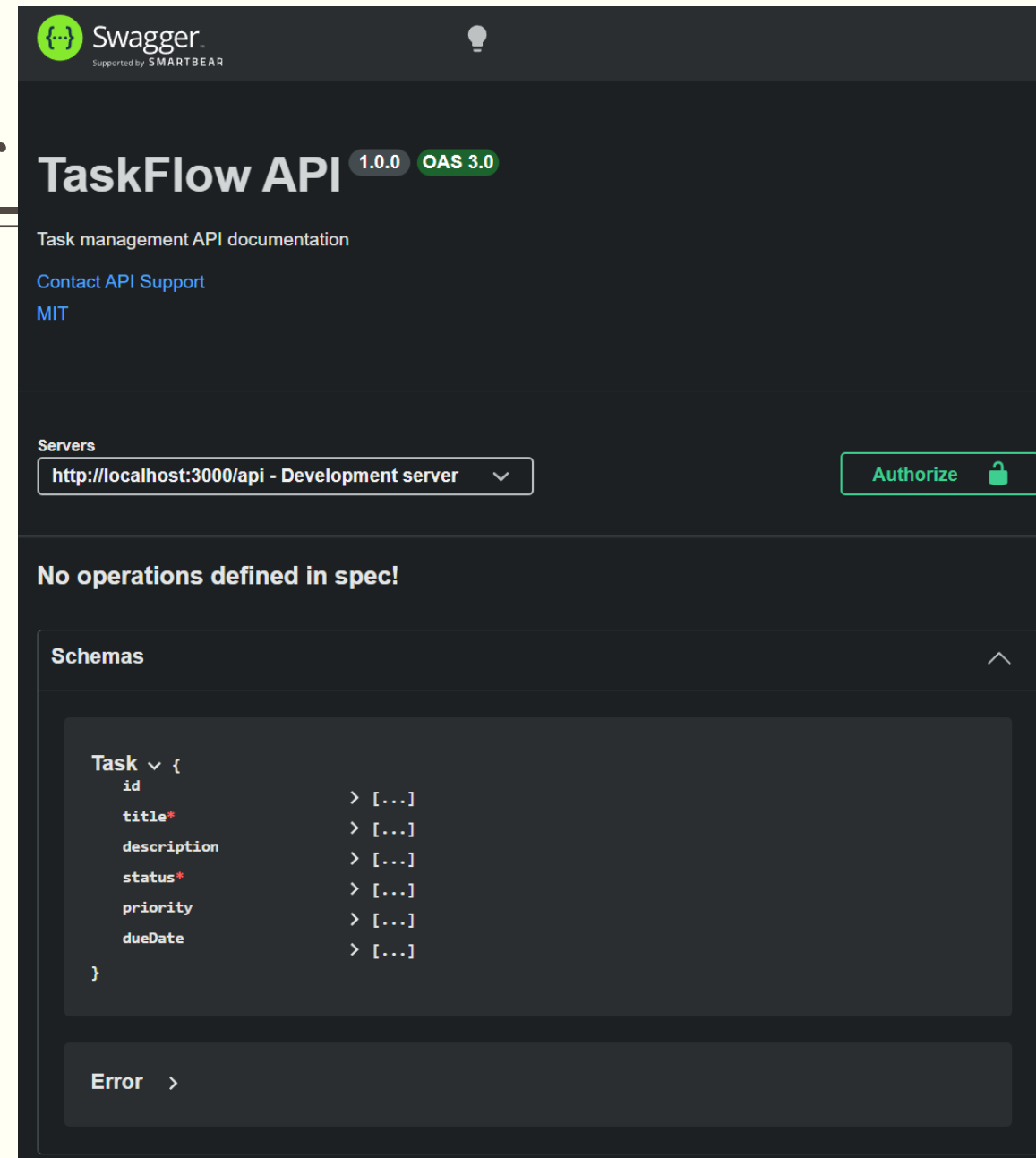
// Специфікація OpenAPI
const swaggerOptions = {...}

const swaggerSpec = swaggerJsdoc(swaggerOptions);

// Публікація Swagger UI
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));

const PORT = 3000;

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
  console.log(`Swagger UI available at http://localhost:${PORT}/api-docs`);
});
```



```
// Стандартизовані відповіді з помилками покращують взаємодію з клієнтом
class ApiError extends Error {
  constructor(message, statusCode, code = null, details = null) {
    super(message);
    this.statusCode = statusCode;
    this.code = code;
    this.details = details;
    this.isOperational = true;
    this.timestamp = new Date().toISOString();

    Error.captureStackTrace(this, this.constructor);
  }
}

class ValidationError extends ApiError {
  constructor(errors) {
    super('Validation failed', 400, 'VALIDATION_ERROR', errors);
  }
}

class NotFoundError extends ApiError {
  constructor(resource = 'Resource') {
    super(`${resource} not found`, 404, 'NOT_FOUND');
  }
}

class ConflictError extends ApiError {
  constructor(message = 'Resource conflict') {
    super(message, 409, 'CONFLICT');
  }
}
```

## Централізована модель обробки помилок

- Помилки не повинні оброблятися хаотично в кожному маршруті, а мають проходити через єдиний контрольований канал, який формує стандартизовану відповідь.
  - Першим кроком є введення базового класу `ApiError`, який розширює стандартний `Error` і доповнює його атрибутами, необхідними для API: HTTP-статусом, прикладним кодом помилки, додатковими деталями та часовою позначкою.
  - Це дозволяє відокремити технічні помилки від бізнес-логіки і забезпечити єдину структуру відповіді.
  - На основі цього класу створюються спеціалізовані типи помилок: `ValidationError`, `NotFoundError`, `ConflictError`, які відображають різні сценарії відмови.

```
// Middleware обробки помилок
function apiErrorHandler(err, req, res, next) {
  let error = { // Помилка за замовчуванням
    success: false,
    error: {
      message: 'Internal server error',
      code: 'INTERNAL_ERROR',
      timestamp: new Date().toISOString(),
      requestId: req.requestId,
    },
  },
};

let statusCode = 500;
// Відомі прикладні помилки API
if (err instanceof ApiError) {
  statusCode = err.statusCode;
  error.error = {
    message: err.message,
    code: err.code,
    details: err.details,
    timestamp: err.timestamp,
    requestId: req.requestId,
  };
}
else if (err.name === 'ValidationError') {
  statusCode = 400; // Помилки валідації Mongoose

  const details = Object.values(err.errors).map((e) => ({
    field: e.path,
    message: e.message,
    value: e.value,
  }));

  error.error = {
    message: 'Validation failed',
    code: 'VALIDATION_ERROR',
    details,
    timestamp: new Date().toISOString(),
    requestId: req.requestId,
  };
}
}
```

# Централізована модель обробки помилок

Функція `apiErrorHandler()` розрізняє кілька класів ситуацій.

- Якщо це власна `ApiError`, відповідь формується безпосередньо з її властивостей.
- Якщо це помилка валідації `Mongoose`, вона перетворюється на масив структурованих деталей із полями `field`, `message`, `value`.
- Якщо це дубль ключа `MongoDB` (11000), сервер повертає конфлікт 409 із вказанням конкретного поля та значення.
- Якщо помилка пов'язана з `JWT`, клієнт отримує одну з двох чітких причин: `INVALID_TOKEN` або `TOKEN_EXPIRED`.
- Для всіх інших, неочікуваних збоїв, інформація журналюється, а внутрішні деталі приховуються в `production`.

@Марченко С.В., ЧДБК, 2026

```
else if (err.code === 11000) {
  statusCode = 409; // Дублювання ключа в MongoDB
  const field = Object.keys(err.keyPattern)[0];
  const value = err.keyValue[field];
  error.error = {
    message: `Duplicate value for ${field}: ${value}`,
    code: 'DUPLICATE_KEY', details: { field, value },
    timestamp: new Date().toISOString(),
    requestId: req.requestId,
  };
}
// Помилки JWT
else if (err.name === 'JsonWebTokenError') {
  statusCode = 401; error.error = {
    message: 'Invalid token', code: 'INVALID_TOKEN',
    timestamp: new Date().toISOString(),
    requestId: req.requestId,
  };
}
else if (err.name === 'TokenExpiredError') {
  statusCode = 401; error.error = {
    message: 'Token expired', code: 'TOKEN_EXPIRED',
    timestamp: new Date().toISOString(),
    requestId: req.requestId,
  };
}
else { // Журналювання неочікуваних помилок
  console.error('Unexpected error:', {
    message: err.message, stack: err.stack,
    url: req.url, method: req.method,
    user: req.user?.id,
    timestamp: new Date().toISOString(),
  });
  // У production не розкриваємо внутрішні деталі
  if (process.env.NODE_ENV !== 'production') {
    error.error.details = {
      message: err.message, stack: err.stack,
    };
  }
}
// Надсилання відповіді з помилкою
res.status(statusCode).json(error);
}
```

```
// Обгортка для валідації запиту
function validateRequest(schema) {
  return (req, res, next) => {
    const { error, value } = schema.validate(req.body, {
      abortEarly: false,
      stripUnknown: true,
    });

    if (error) {
      const errors = error.details.map((detail) => ({
        field: detail.path.join('.'),
        message: detail.message,
        type: detail.type,
      }));

      return next(new ValidationError(errors));
    }

    // Замінюємо тіло запиту валідованими даними
    req.body = value;
    next();
  };
}
```

- `validateRequest()` обгортає Joi-схему і перетворює результати її перевірки у `ValidationError`.

- Валідація запиту стає частиною загального конвеєра обробки помилок. Після успішної перевірки `req.body` замінюється вже очищеними й валідованими даними, а при помилці клієнт одразу отримує структуровану відповідь із переліком проблемних полів.
- Наведений приклад `taskSchema` і маршруту `POST /api/tasks` демонструє, що маршрут лишається коротким, а вся логіка некоректного вводу централізовано переводиться в узгоджений формат відповіді.

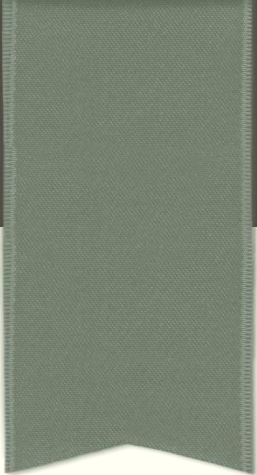
## Централізована модель обробки помилок

```
// Приклад використання зі схемою Joi
const Joi = require('joi');

const taskSchema = Joi.object({
  title: Joi.string().min(3).max(255).required(),
  description: Joi.string().max(2000).optional(),
  priority: Joi.string().valid('low', 'medium', 'high').default('medium'),
  dueDate: Joi.date().iso().min('now').optional(),
  assigneeId: Joi.string().pattern(/^[\0-9a-fA-F]{24}$/).optional(),
});

app.post('/api/tasks', validateRequest(taskSchema), async (req, res, next) => {
  try {
    const task = await createTask(req.body);

    res.status(201).json({
      success: true,
      data: task,
    });
  } catch (error) {
    next(error);
  }
});
```



# **ДЯКУЮ ЗА УВАГУ!**

**Наступне питання: Моделювання даних та інтеграція бази даних.**